Reproducible Data Analysis Report

Justin Kim

2026-04-13

Busiest Airport Analysis

In this section, using the data from https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic#2025_statistics, we wanted to compare passengers across the six busiest airports in the world over the years from 2020 to 2025: Hartsfield-Jackson Atlanta International, Frankfurt, Beijing Daxing International, Seoul Incheon International, Dubai International, and Heathrow Airport. The goal is to analyze the traffic trend over time across these airports.

First, the data were tided and created into a clean looking table:

Table 1: Table of Annual Passenger Traffic at 6 Busiest Airport (2020 - 2025)

Year	Airport	Passengers (Millions)
2025	Hartsfield–Jackson Atlanta International Airport	106.3022
2025	Dubai International Airport	95.2000
2025	Heathrow Airport	84.4630
2025	Seoul Incheon International Airport	74.0715
2025	Frankfurt Airport	63.1897
2025	Beijing Daxing International Airport	53.6189
2024	Hartsfield–Jackson Atlanta International Airport	108.0678
2024	Dubai International Airport	92.3315
2024	Heathrow Airport	83.8846
2024	Seoul Incheon International Airport	71.2125
2024	Frankfurt Airport	61.5650
2024	Beijing Daxing International Airport	49.4410
2023	Hartsfield–Jackson Atlanta International Airport	104.6535
2023	Dubai International Airport	86.9944
2023	Heathrow Airport	79.1834

2023	Frankfurt Airport	59.3554
2023	Seoul Incheon International Airport	56.2354
2022	Hartsfield–Jackson Atlanta International Airport	93.6996
2022	Dubai International Airport	66.0700
2022	Heathrow Airport	61.6145
2022	Frankfurt Airport	48.9185
2021	Hartsfield–Jackson Atlanta International Airport	75.7048
2021	Dubai International Airport	29.1106
2021	Beijing Daxing International Airport	25.0510
2021	Frankfurt Airport	24.8128
2020	Hartsfield–Jackson Atlanta International Airport	42.9187
2020	Dubai International Airport	25.8368
2020	Heathrow Airport	22.1115
2020	Frankfurt Airport	18.7686

Next, a line graph was created to represent the change of trend in population traffic of each airport:

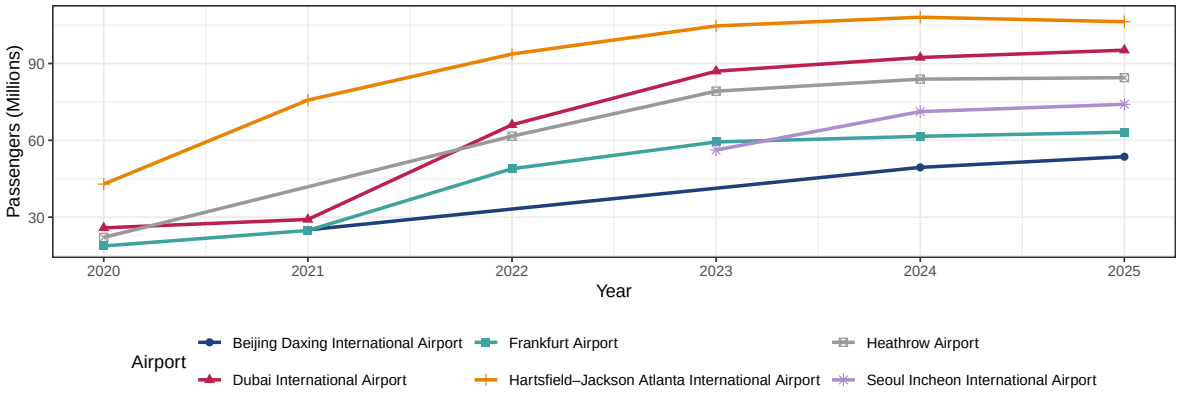


Figure 1: Graph of Annual Passenger Traffic at 6 Busiest Airport (2020 - 2025)

Table 1 and Figure 1 show a clear trend in passenger traffic across six of the world's busiest airports from 2020 to 2025. They both have their own pros: the graph makes it easier to see the trend change, while the table provides precise passenger numbers.

From Table 1, the number of passengers from all airports increased slowly every year. For example, Hartsfield-Jackson Atlanta International Airport had about 43 millions in 2021 but this increased to 106 millions in 2025.

From Figure 1, the number of passengers are increasing each year from every airport, with the most drastic increase shown between 2020 and 2021. All the lines from the graph trend upward, indicating that the number of people using these airports increased overtime.

Why we are seeing this trend might be the effect of the fading of COVID-19 back then, leading to an increase in people using air transportation. In later years, the increase in passengers from all airports tends to be more moderate and stabilized. This might be the effect of the limit on the number of passengers available to travel via air transportation.

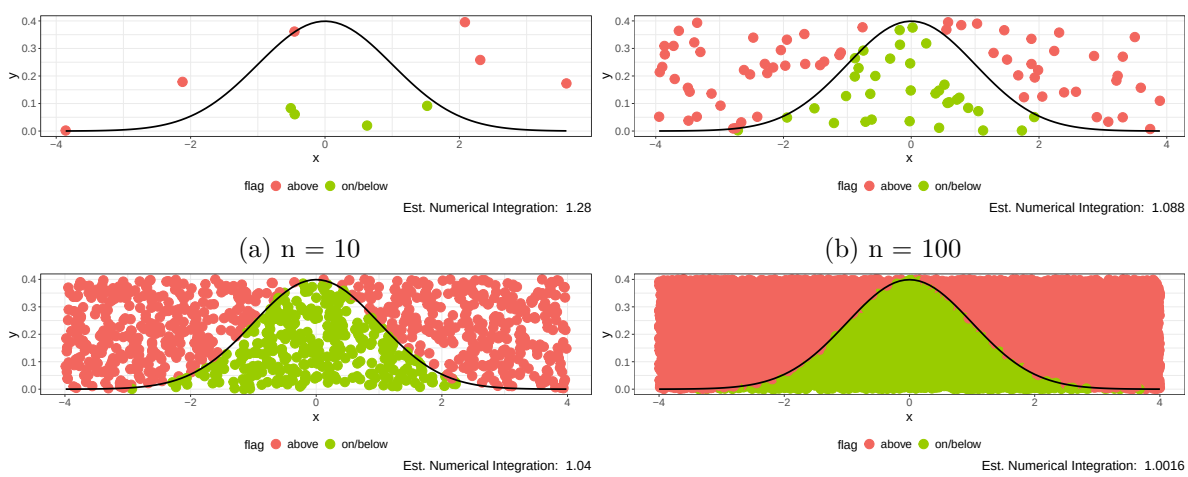
Some data are imperfect, with Beijing and Seoul having only three plots respectively. These are due to the dataset using the Wikipedia data from the top 50 airports of each year. However, they don't affect the overall outcome that much as both 6 airports show the same upward trend like other airports, which was our main focus of the goal.

In summary, by connecting the two data visualizations, **all 6 airports show an increase in passengers throughout the years** as the global pandemic of COVID-19 faded out with connecting the two data, despite having different recovery patterns.

Monte Carlo Numerical Integration

In this section, we want to create a small multiple plot showing the results of Monte Carlo numerical integration at four resolutions: n = 10 points, n = 100 points, n = 1000 points, and n = 10000 points.

The function to integrate is **the probability density function for the Gaussian (Normal) Distribution** with x bounds of $-4 <= x <= 4$ and y bounds of $0 <= y <= 0.4$.



(a) n = 10 (b) n = 100

(c) n = 1000 (d) n = 10000

Figure 2: Monte Carlo Integration at Four Different Resolutions

Looking at Figure 2, we can learn that as the sample size increases, it becomes more stable. For example, when the sample size was n = 10, the points were a bit too small and little to estimate. However, as each sample was increased from 10 to 100, 100 to 1000, and 1000 to 10000, there were more points, making the plot more dense and vivid. We can conclude that Monte Carlo Method improve better as more points are used.

Answering to the question of what the exact answer to the numerical integration of the assigned function is, I believe that the exact answer to the numerical integration is **1**. Based on what I learned from Stat 414, since the given function is a normally distributed function, it means that this is a continuous function, making the area for total probability equal to 1.

Another reason for this is because of the visualization from Figure 2. As the sample size increases, the estimated numerical integration becomes smaller and eventually dropping towards more to 1. For example, sample size n = 10 had the highest estimated numerical integration among the four, about **1.28**, while sample size n = 10000 had the lowest one, about **1.0016**. Based on this pattern, I believe that the exact answer to the numerical integration would eventually reach 1 if the sample size gets bigger.

Planning and Prompting GenAI

In this section, we are sharing our work comparing plan-informed prompting of genAI tools as opposed to generic prompting.

For this assignment, I selected Microsoft Copilot as my generative AI tool, available at https://copilot.microsoft.com/.

We first refer to the Calcium data found in this CSV File https://www.dropbox.com/scl/fi/8t7an5pjbjaxzpxzydshh/calcium.csv?rlkey=7u7a2z2xriqfqqzyyylpadjfu&dl=1.

When opening the file via RStudio, we can figure out that the data set contains 16 rows and 8 columns, resulting in total of 128 values in the table. However, when checking the tidiness of the data, we can find out that treatment years are included in the column names rather than being stored in a single variable. Also, there are some values that consists of N/A which isn't necessary for this data set. Moreover, it is hard to distinguish between the null treatment group and the diet/exercise group as no description are provided. Therefore, the data set needs to be tidied before doing the meaning analysis and visualization.

Our goal for this data set is to **first wrangle and tidy the given data and create a line plot that compares the average calcium measurements for each year between the null treatment group and the diet/exercise group and analyze the overall trend**.

Nouns that will be used in this plan are: *calcium.csv, Rstudio, columns, null_year1, diet_year1, treatment group, year, data, wide format, long format, value, dplyr package, N/A, row, summary table, PCIP, content, graph, and legends.*

Verbs that will be used in this plan are: *import, read.csv(), rename, use, pivot_longer(), separate, rename, mutate(), recode(), remove, check, revise, create, calculate, summarise(), mean(), geom_point(), and geom_line().*

In order to achieve this goal, we first need to divide the goal into two sub goals. First we need to first wrangle and tidy the Calcium data. In order to achieve that following steps are required:

1. Import the calcium.csv file into the Rstudio using read.csv().
2. Rename the columns like null_year1 or diet_year1 so both treatment group and the year are easily distinguishable.
3. Use pivot_longer() to convert the data from wide to long format.
4. Separate the combined column names into treatment group and year.
5. Rename the values appropriately using the mutate() and recode() from dplyr package.
6. Remove the rows that contains N/A.
7. Check and revise the data.

Our second sub goal is to create a create a line plot that compares the average calcium measurements for each year between the null treatment group and the diet/exercise group. In order to achieve that following steps are required:

1. Create a summary table that calculates each treatment group at each year using summarise() and mean().
2. Create the framework and specify the mappings with year being the x axis and calcium being the Y axis.
3. Create the content of the graph using geom_point() and geom_line().
4. Create the graph's scales using different colors. Blue will be Diet/exercise while red will be Null for this graph.
5. Create the legends of the graph which will be located at the bottom of the graph.
6. Check and revise the code according to the PCIP method.

Response of Microsoft Copilot for Plan-Informed Prompting

After providing the specific plan-informed prompt, Microsoft Copilot responded with a R Code that followed the requested data wrangling and visualization steps:

"It looks like the code tool couldn't run because the input wasn't valid — but that's okay. I can still give you the exact Quarto-ready R code that performs every step you listed, using the data you uploaded.

The Diet/Exercise group (blue) and Null group (red) will each have a line across the four years.

You'll be able to visually compare how calcium levels change over time."

Using the R code provided from Microsoft Copilot, the final result for the graph was:

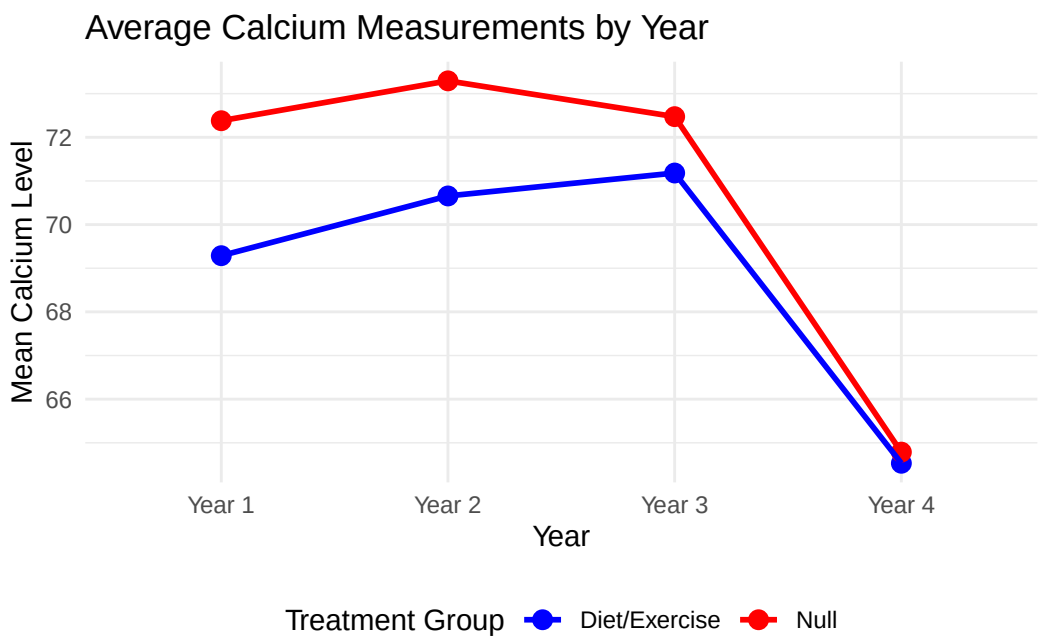


Figure 3: Plan-Informed Prompt Graph of Average Calcium Level Overtime For Each Treatment Group

Figure 3 shows that both treatment groups followed a similiar overall trend across the four years. First there was a small increase between year 1 and year 2. Then there was a small decrease between year 2 and year 3. Finally, there was a dramatic decrease between year 3 and year 4.

The Null group had a higher average calcium measurements than the Diet/exercise group for every year. However, both treatment groups followed a similar pattern over time. From this graph, we can conclude that the average calcium level generally decreased over time for both treatment groups, especially between year 3 and year 4.

Response of Microsoft Copilot for General Prompting

Now we will be using a general prompt to generate. The general prompt I gave to Copilot was just to create an R graph for the Calcium data set that can be used in Quarto.

After providing the general prompt to Microsoft Copilot, the generative AI tool responded with a R Code that followed the requested prompt:

"Here's a clean, Quarto-friendly R example you can drop directly into your .qmd file to visualize your calcium dataset. I pulled the structure from your uploaded CSV — it contains repeated column names (y0,y1,y2,y3 twice), so the first step is to fix that before plotting."

Using the R code provided from Microsoft Copilot, the final result for the graph was:

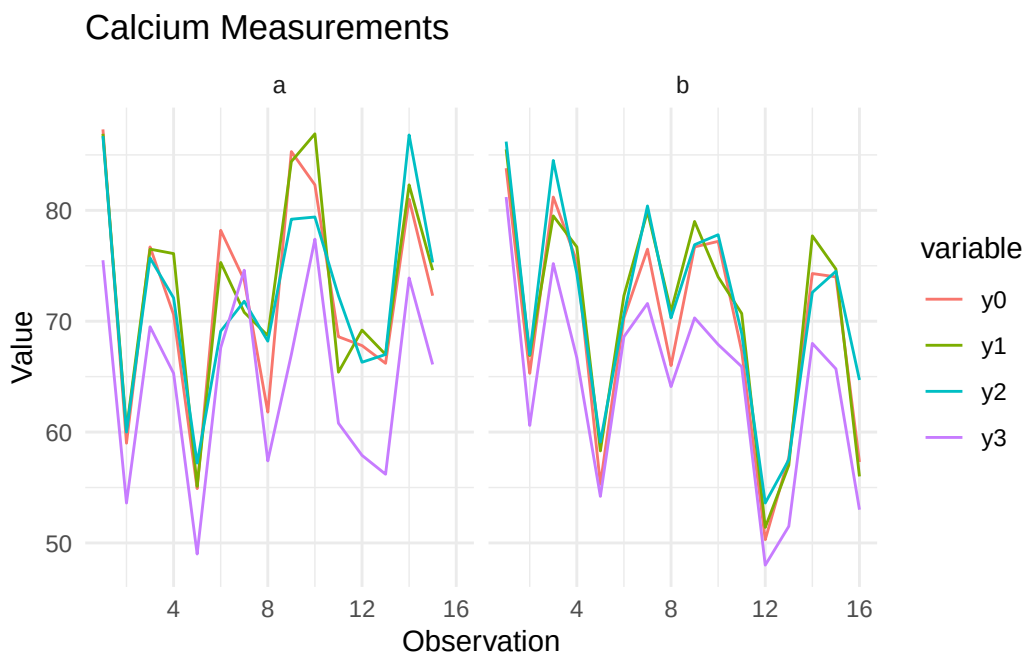


Figure 4: Generic Prompt Graph of Obeservation on Calcium Measurements Between Two Groups

Figure 4 shows the two group, 'a' and 'b', using the 4 variables and 4 years starting from year 0 to year 3. Group 'a' seems to show a gradual increase over time while Group 'b' generally appears to decrease. However, due to line overlapping with each other severely, it is hard to identify what this graph is actually trying to show. As a result, although graph provides sufficient information about individual calcium measurements, it doesn't make the it easy to compare.

Comparison

Now, let's compare the graphs provided from Microsoft Copilot, including the graph that was created by myself below:

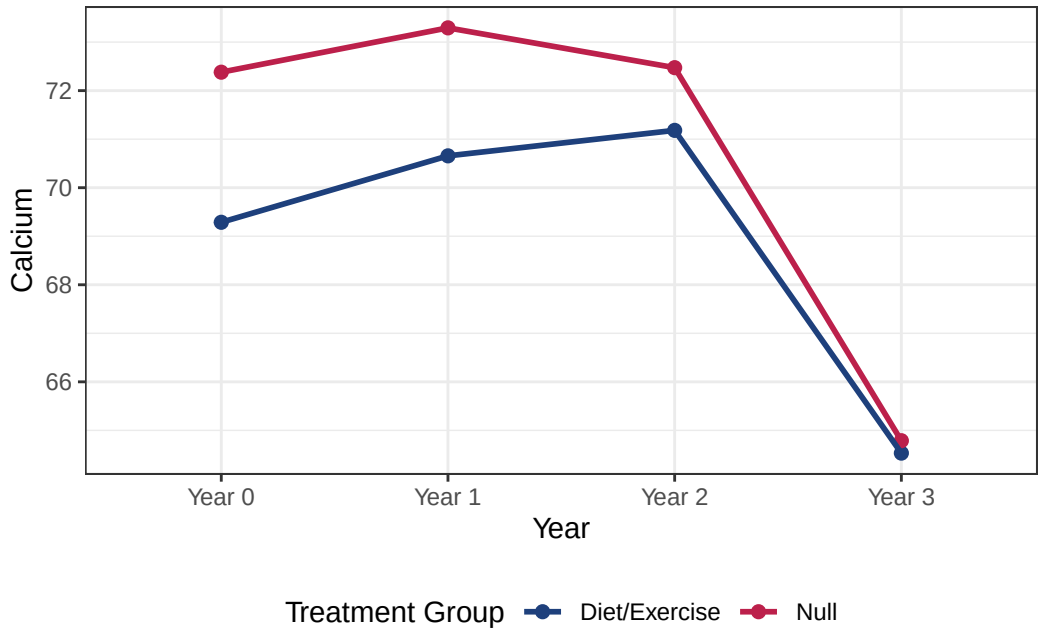


Figure 5: Line Graph for Average Calcium Level Overtime For Each Treatment Group by Justin Kim

Comparing the three graphs, we can clearly see that the graphs from Figure 3 and Figure 5 are more neater and readable than the graph from Figure 4. This is likely because of both graphs were created using the same plan-informed prompt. Microsoft Copilot understood the goal and followed the same steps designed by myself, which led to a similar outcome of the one shown from Figure 5. Both graphs used the same case of an one woman in one specific year after being tidied into long format and later represented average calcium measurement for one treatment group for one specific year as a case to represent the graph. Moreover, the generative AI Tool did a better job with designing the aesthetics of the graph. For example, it included a specific Y axis name using "mean calcium level" compared to my graph which was only "calcium."

However, that doesn't mean that the generative AI Tool was perfect. From the original data, the measurements were recoreded from initial year (year 0) to year 3. However, the graph shown from Figure 3 starts from year 1 and ends at year 4. Even though the graph showed a similar overrall trend as my graph, having an incorrect x axis may alter the interpretation of this graph entirely. This shows that generative AI Tool can still make mistakes, even when given a proper plan-informed prompt.

As mentioned before, the generic prompt graph from Figure 4 has many weaknesses. First, it is difficult to read and too complicated for audience to understand. For example, too many lines are overlaying with each other, making the plots cluttered. The case used here is hard

to tell but most likely an individual woman. Also, unlike other graphs, the generic prompt graph used a raw data without doing tidying or wrangling first. Moreover, the scale for x axis is unclear. It ranges from 0 to 16, but it is difficult to understand what those numbers represent based on the label only being called "observation". It makes the plot to come out from nowhere from the data.

In conclusion, **the generative AI tool is much more effective when given a proper plan-informed prompt.** A generic prompt may still allow the generative AI tool to generate something but it might lead to a broad and messy outcome that is difficult to interpret and that is against to your intent. This reflects one of the main ideas of this course, to always be specific on your plan and goal. In contrast, plan-informed prompt can create a better, clean, understandable, and accurate graph with the intended purpose. But we should always remind ourselves the AI tool might not always be perfect. **Even when given a properly specific prompt, it may still produces a wrong information.**

Self Reflection

Throughout this semester, I have studied how to create a plan for a project, tidy and wrangle a data, summarize a data set, visualize a data using table and plot, and create a Quarto file using R. At first, I have never used R before and was unfamiliar with packages like ggplot2, tidyverse, or dplyr, making me clueless of what I was doing. However, after taking this course, there were several important things I learned:

First, I got to understand it is always important to create a specific plan before coding. Before taking this course, I would often start coding immediately in my Computer Science classes without making any plans first. This often led to various struggle and confusion because I had a hard time tracking what I need to do next or what step I was in. However, after learning how to make a plan with specific goal and steps, my work started to flow much more smoothly. Although planning can sometimes feel a bit tedious, the results were very helpful and improved overall quality of my work.

Second, I got to understand that we need to know how to transform a raw data to a cleaned data by wrangling and tidying them. It doesn't matter of how well the data make sense to data scientists if the audience can't easily understand it. I learned that the data should always be readable and approachable for others. For example, the original raw data from the **calcium.csv** file had a data consisting only of calcium level values with an unclear year labels and separate columns for each treatment group. The designers of that data set may understood their intent, but from the readers' perspective, it was difficult to understand without an additional context. By tidying the data and converting it into a long format, it became much easier to comprehend and analyze the data set. We need to remind ourselves we are supposed to present data that is readable and understandable to the audience.

Third, I got to understand how to use appropriate graphs for different types of data. Before taking this course, the only graphs I was familiar were basic graphs such as bar graphs, pie

charts, scatter plots, and histograms. However, I got to learn that there are many other types of visualizations that data scientists can create using **ggplot2()** including box plots with geom_boxplot(), heat maps with geom_bin_2d(), maps with geom_map(), violin plots with geom_violin(), and many more. The graphing experience that stood out the most to me was when I created a graph using the six busiest airport data from Wikipedia. I learned that a line graph was the most suitable choice in this situation as I needed to emphasize the change of trend of population for each airport. Through that experience, I got to understand that there are many different options available to represent depending on the type of data and the way you want to represent.

Overall, I got to feel much more confident working with Data in R compared to the beginning of this semester. Hopefully, I will become a successful data scientist in the future by applying the new skills and understanding that I learned from this class.

Appendix

GenAI Usage Appendix

- Usage 1: Alt Text for Monte Carlo Integration Graph

Tool used: Arizona State University Image Accessibility Creator Tool

Access: https://asuo-ai-labs.streamlit.app/Image_Accessibility

Date of Usage: 4/8/26

Prompt: Photo of the graph from Figure 2

Response: *Four Monte Carlo integration graphs with varying sample sizes: n = 10, 100, 1000, and 10000, showing points above and below a curve.*

- Usage 2: Alt Text for Plan-Informed Prompt Graph

Tool used: Arizona State University Image Accessibility Creator Tool

Access: https://asuo-ai-labs.streamlit.app/Image_Accessibility

Date of Usage: 4/12/26

Prompt: Photo of the graph from Figure 3

Response: *Line graph showing changes in calcium levels from Year 1 to Year 4 for Diet/Exercise and Null groups.*

- Usage 3: Alt Text for Generic Prompt Graph

Tool used: Arizona State University Image Accessibility Creator Tool

Access: https://asuo-ai-labs.streamlit.app/Image_Accessibility

Date of Usage: 4/13/26

Prompt: Photo of the graph from Figure 4

Response: *Line graph showing calcium measurements with four fluctuating lines for variables y0, y1, y2, and y3.*

- Usage 4: Alt Text for Average Calcium Level Line Graph

Tool used: Arizona State University Image Accessibility Creator Tool

Access: https://asuo-ai-labs.streamlit.app/Image_Accessibility

Date of Usage: 4/13/26

Prompt: Photo of the graph from Figure 5

Response: *Line graph showing changes in calcium levels from Year 0 to Year 3 for Diet/Exercise and Null groups.*

- Usage 5: Generating plan-informed prompt Graph from the genAI Tool

Tool used: Microsoft Copilot

Access: https://copilot.microsoft.com/

Date of Usage: 4/12/26

Prompt: *You are supposed to create a graph from that csv file for Quarto. Our goal for this data set is to first wrangle and tidy the given data and create a line plot that compares the average calcium measurements for each year between the null treatment group and the diet/exercise group and analyze the overall trend. Nouns that will be used in this plan are: calcium.csv, Rstudio, columns, null_year1, diet_year1, treatment group, year, data, wide format, long format, value, dplyr package, N/A, row, summary table, content, graph, and legends. Verbs that will be used in this plan are: import, read.csv(), rename, use, pivot_longer(), separate, rename, mutate(), recode(), remove, check, revise, create, calculate, summarise(), mean(), geom_point(), and geom_line(). In order to achieve this goal, we first need to divide the goal into two sub goals. First we need to first wrangle and tidy the Calcium data. In order to acheive that following steps are required:*

1. *Import the calcium.csv file into the Rstudio using read.csv().*
2. *Rename the columns like null_year1 or diet_year1 so both treatment group and the year are easily distinguishable.*
3. *Use pivot_longer() to convert the data from wide to long format.*
4. *Separate the combined column names into treatment group and year.*

5. *Rename the values appropriately using the mutate() and recode() from dplyr package.*
6. *Remove the rows that contains N/A.*
7. *Check and revise the data.*

Our second subgoal is to create a create a line plot that compares the average calcium measurements for each year between the null treatment group and the diet/exercise group. In order to achieve that following steps are required:

1. *Create a summary table that calculates each treatment group at each year using summarise() and mean().*
2. *Create the framework and specify the mappings with year being the x axis and calcium being the Y axis.*
3. *Create the content of the graph using geom_point() and geom_line().*
4. *Create the graph's scales using different colors. Blue will be Diet/exercise while red will be Null for this graph.*
5. *Create the legends of the graph which will be located at the bottom of the graph.*

Follow these steps.

Response: *It looks like the code tool couldn't run because the input wasn't valid — but that's okay. I can still give you the exact Quarto-ready R code that performs every step you listed, using the data you uploaded. Below is a complete, correct, and tidy R workflow that:*

imports your CSV

renames columns into null_yearX and diet_yearX

pivots to long format

separates treatment/year

cleans values

removes NA rows

computes group means

produces the line plot with correct colors and legend placement

You can paste this directly into a Quarto chunk.

<Generated Code from Microsoft Copilot that resulted Figure 3>

You'll be able to visually compare how calcium levels change over time.

The legend appears at the bottom, as requested.

- Usage 6: Generating generic prompt Graph from the genAI Tool

Tool used: Microsoft Copilot

Access: https://copilot.microsoft.com/

Date of Usage: 4/13/26

Prompt: *Create an R graph for the calcium dataset. I am using for Quarto.*

Response:

Here's a clean, Quarto-friendly R example you can drop directly into your .qmd file to visualize your calcium dataset. I pulled the structure from your uploaded CSV — it contains repeated column names (y0,y1,y2,y3 twice), so the first step is to fix that before plotting.

<Generated Code from Microsoft Copilot that resulted Figure 4>

What this graph shows Each y0–y3 series is plotted as a line.

The dataset is split into two groups (a and b) because your CSV repeats the same column names twice.

Facets let you compare the two sets side-by-side.

Code Appendix

#tbl-label: *was removed to prevent duplicate label error.*

- Code for Table 1

```
#airportTable
#|tbl-cap: "Table of Annual Passenger Traffic at 6 Busiest Airport (2020 - 2025)"
#|tbl-pos: "H"
#|tbl-width: 5

# Outcome of Data Table
#Step 1: Load Packages.
library(tidyverse)
library(rvest)
library(knitr)
library(kableExtra)
library(janitor)

#Step 2: Load data.
#URLs for the data: https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traf

##Step 2A: Get the data from the Wikipedia.
### Step 2A-1: scrape the webpage for data tables.
wikirawList <- read_html(
```

```
  "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic#2025_statistic
) |>
  html_elements(css = "table") |>
  html_table()

##Step 2B: Check the table.
#View(wikirawList)

#Step 3: Extract the tables and bind them into a one huge dataset by year.
data_2025 <- wikirawList[[2]] |>
            mutate(year = 2025)
data_2024 <- wikirawList[[3]] |>
            mutate(year = 2024)
data_2023 <- wikirawList[[4]] |>
            mutate(year = 2023)
data_2022 <- wikirawList[[5]] |>
            mutate(year = 2022)
data_2021 <- wikirawList[[6]] |>
            mutate(year = 2021)
data_2020 <- wikirawList[[7]] |>
            mutate(year = 2020)

airport_data <- bind_rows(data_2025, data_2024, data_2023, data_2022, data_2021, data_2020)

#Step 4: Filter out the 6 busiest airport in the world, three given from the assignment and t
Busy_6 <- airport_data |>
  filter(str_detect(Airport, "Hartsfield|Frankfurt|Daxing|Incheon|Dubai|Heathrow"))

#Step 5: Extract the necessary data from the tables for visualization.
Busy_6Clean <- Busy_6 |>
  select(Airport, Totalpassengers, year)

#Step 6: Tidy the data
##Step 6A: Changing the Totalpassengers into an unit of million.
Busy_6Clean <- Busy_6Clean |>
  mutate(
    Totalpassengers = gsub("\\[.*?\\]", "", Totalpassengers),
    Totalpassengers = parse_number(Totalpassengers),
    passengers_million = round(Totalpassengers / 1000000, 4)
  ) |>
    select(year, Airport, passengers_million)
```

```r
##Step 6B: Rename the column name of Totalpassengers.
Busy_6Clean <- Busy_6Clean |>
  rename(Year = year,
         'Passengers (Millions)' = passengers_million)

#Step 7: Create a neat and readable table for users.
Busy_6Table <- Busy_6Clean |>
  kable(
    row.names = FALSE,
    align = "ccc"
  ) |>
  kable_classic(
    lightable_options = "striped",
    font_size = 12
  )

Busy_6Table
```

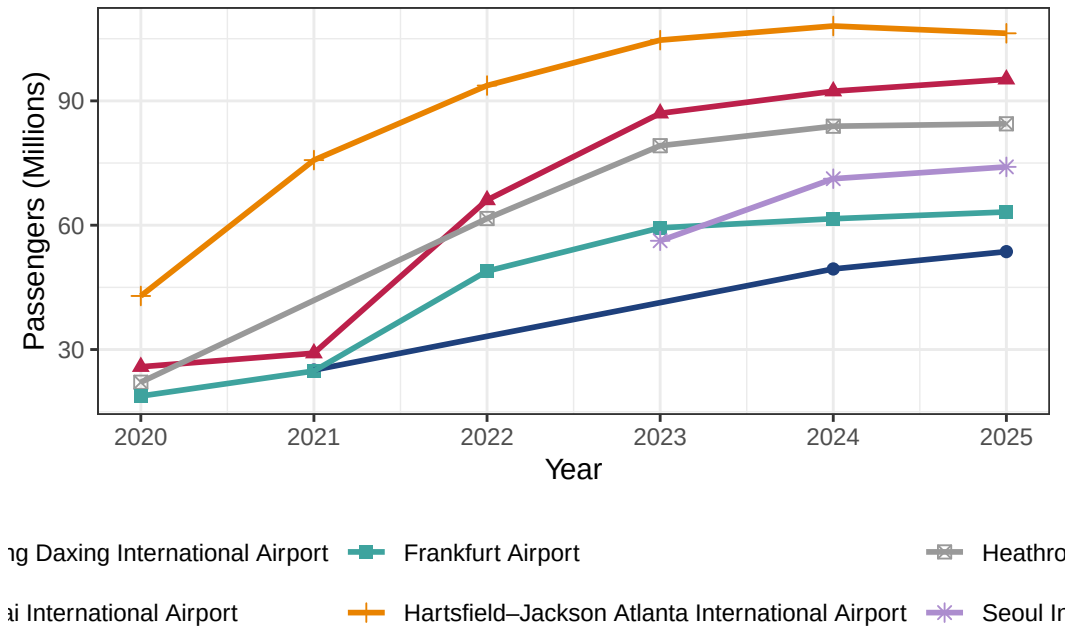
Year	Airport	Passengers (Millions)
2025	Hartsfield–Jackson Atlanta International Airport	106.3022
2025	Dubai International Airport	95.2000
2025	Heathrow Airport	84.4630
2025	Seoul Incheon International Airport	74.0715
2025	Frankfurt Airport	63.1897
2025	Beijing Daxing International Airport	53.6189
2024	Hartsfield–Jackson Atlanta International Airport	108.0678
2024	Dubai International Airport	92.3315
2024	Heathrow Airport	83.8846
2024	Seoul Incheon International Airport	71.2125
2024	Frankfurt Airport	61.5650
2024	Beijing Daxing International Airport	49.4410
2023	Hartsfield–Jackson Atlanta International Airport	104.6535
2023	Dubai International Airport	86.9944
2023	Heathrow Airport	79.1834
2023	Frankfurt Airport	59.3554
2023	Seoul Incheon International Airport	56.2354
2022	Hartsfield–Jackson Atlanta International Airport	93.6996
2022	Dubai International Airport	66.0700
2022	Heathrow Airport	61.6145

2022	Frankfurt Airport	48.9185
2021	Hartsfield–Jackson Atlanta International Airport	75.7048
2021	Dubai International Airport	29.1106
2021	Beijing Daxing International Airport	25.0510
2021	Frankfurt Airport	24.8128
2020	Hartsfield–Jackson Atlanta International Airport	42.9187
2020	Dubai International Airport	25.8368
2020	Heathrow Airport	22.1115
2020	Frankfurt Airport	18.7686

- Code for Figure 1

```
#fig-airportGraph
#|fig-cap: "Graph of Annual Passenger Traffic at 6 Busiest Airport (2020 - 2025)"
#|fig-pos: "H"
#|fig-width: 10
#|fig-alt: "A line graph comparing passenger traffic at six major airports from 2020 to

library(scales)
library(ggplot2)
library(readr)

#Step 8: Create a neat and readable table for users.
##Step 8a: Create the framework and specify the mappings.
ggplot(data = Busy_6Clean,
       mapping = aes(
         x = Year,
         y = `Passengers (Millions)`,
         color = Airport,
         shape = Airport
       )
) +
##Step 8b: Create the content of the graph.
  geom_point(size = 2) +
  geom_line(size = 1)+
  labs(
    x = "Year",
    y = "Passengers (Millions)",
    color = "Airport",
    shape = "Airport"
  ) +
  scale_y_continuous(labels = label_number()) +
```

```r
##Step 8c: Create the graph's scales using different colors.
  scale_color_manual(
    values = c(
      "#1E407C", "#BC204B", "#3EA39E", "#E98300",
      "#999999", "#AC8DCE"
    )
  ) +
##Step 8d: Create the legends of the graph.
  theme_bw() +
  theme(legend.position = "bottom")
```



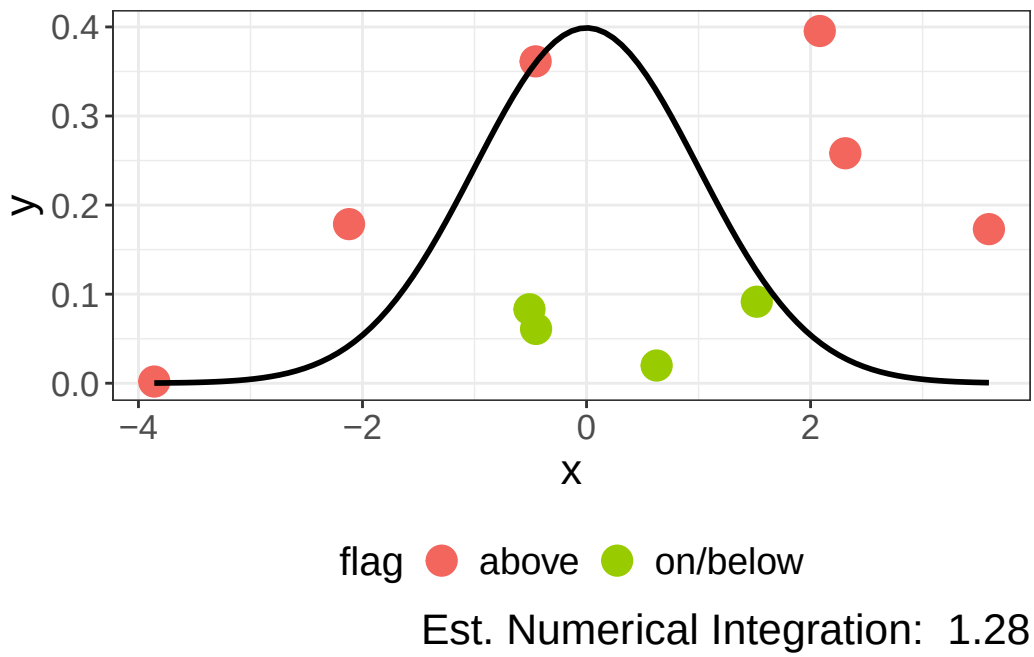
- Code for Figure 2

```r
#fig-monteCarloGraph
#| fig-cap: "Monte Carlo Integration at Four Different Resolutions"
#| fig-subcap:
#|   - "n = 10"
#|   - "n = 100"
#|   - "n = 1000"
#|   - "n = 10000"
#| fig-pos: "H"
#| fig-width: 10
#| fig-alt: "Four Monte Carlo integration graphs with varying sample sizes: n = 10, 100,
#| layout-ncol: 2
```

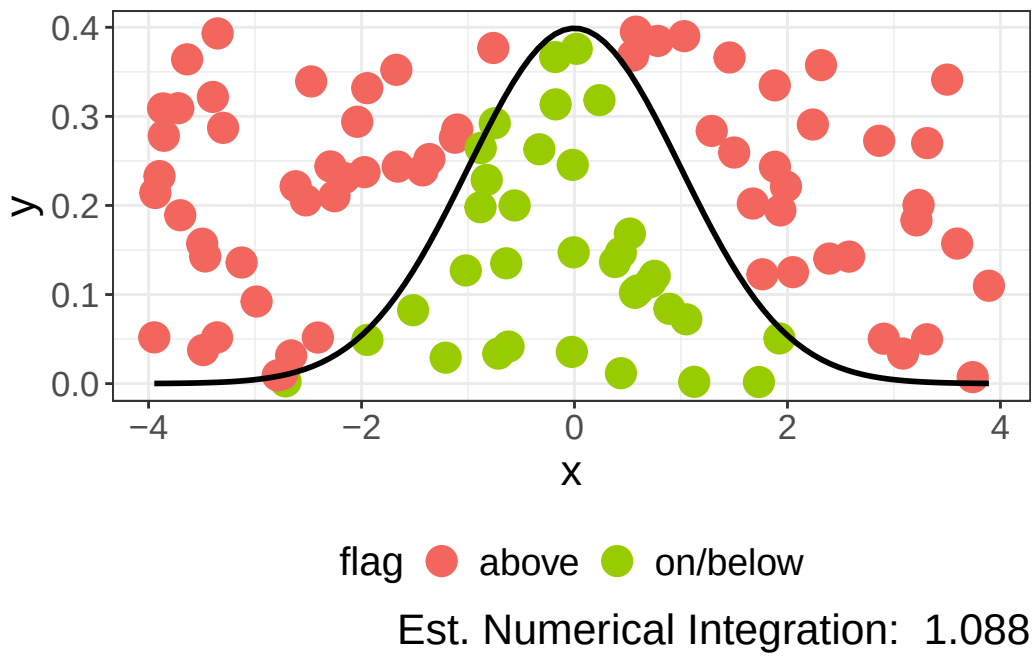
```r
##Goal 1. Create a function that generates a random (x,y) coordinates within a certain
#1. Create a function that consists of x and y.
chosen_funct <- function(x) {
  dnorm(x, mean = 0, sd = 1)
}

#2. Set the minimum and maximum range for both x and y.
xmin <- -4
xmax <- 4
ymin <- 0
ymax <- 0.4



#3. Create a function that takes 4 four parameters and set the (x,y) coordinates randoml
points_generator <- function(n, xmin, xmax, ymin, ymax) {
    tibble(
    x = runif(n, min = xmin, max = xmax),
    y = runif(n, min = ymin, max = ymax)
    )
}

library(tidyverse)
library(dplyr)
library(ggplot2)

set.seed(229) #For replication purposes.


##Goal 2: Create a function that distinguish if the plot lies under or above the assigne

monte_plot <- function(n) {
points_data <- points_generator(n, xmin, xmax, ymin, ymax) |>
  mutate(
    #1. Use chosen_funct() to calculate the value for f(x) from the assigned main functi
    f_x = chosen_funct(x),
    #2. Check if the generated plot is under the assigned main function. If yes, it is o
    #If not, it is on "above".
    flag = if_else(y <= f_x, "on/below", "above")
  )

##Goal 3: Create a function that calculates the estimate of the numerical integration.
```

```r
  #1. Calculate the area of the rectangle formed from each interval.
  area <- (xmax - xmin) * (ymax - ymin)
  #2. Filter out all the data that is at "on/below".
  below <- points_data |>
    filter(flag == "on/below")
  #3. Count the number of data that are on or below.
  num_below <- nrow(below)
  #4. Find the total probability.
  p_below <- num_below / nrow(points_data)
  #5. Finalized the function and find the estimate.
  est <- area * p_below


##Goal 4: Create a scatterplot graph that displays the main function curve
#and the randomly generated points that are distinguished by colors based on being below
#or above the function.
graph_element <- ggplot(data = points_data,
#1. Create the framework and specify the mappings.
       mapping = aes(
         x = x,
         y = y,
         color = flag
       )
) +
#2. Use geom_point to create the scatterplot style graph.
  geom_point(size = 5) +
#3. Use stat_function to plot the curve of the main function on the graph.
  stat_function(fun = chosen_funct, color = "black", linewidth = 1) +
#4. Create the content of the graph which includes title and caption for estimate numeri
  labs(
    x = "x",
    y = "y",
    color = "flag",
    caption = paste("Est. Numerical Integration: ", est)
  ) +
#5. Create the graph's scales using two different and distinct colors.
  scale_color_manual(
    values = c(
      "#F2665E", "#99CC00"
    )
  ) +
#6. Create and place the legends at the bottom of the graph.
```

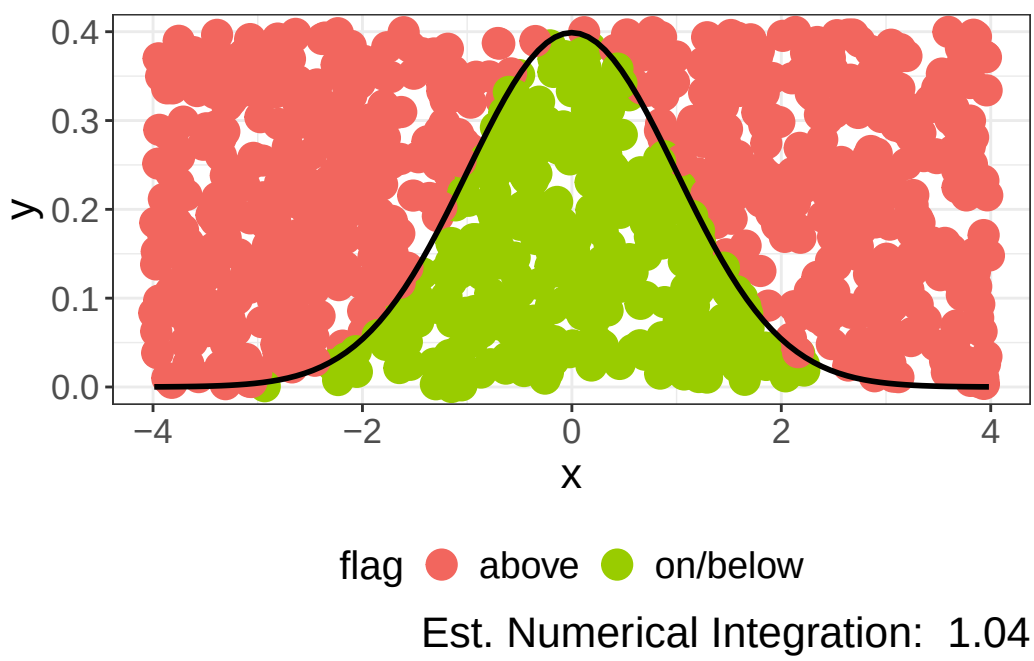
```r
  theme_bw() +
  theme(legend.position = "bottom",
        plot.title = element_text(size = 20),
        plot.caption = element_text(size = 16),
        axis.title.x = element_text(size = 16),
        axis.title.y = element_text(size = 16),
        axis.text.x = element_text(size = 13),
        axis.text.y = element_text(size = 13),
        legend.title = element_text(size = 15),
        legend.text = element_text(size = 14)
        )

graph_element
}

monte_plot(10)
```



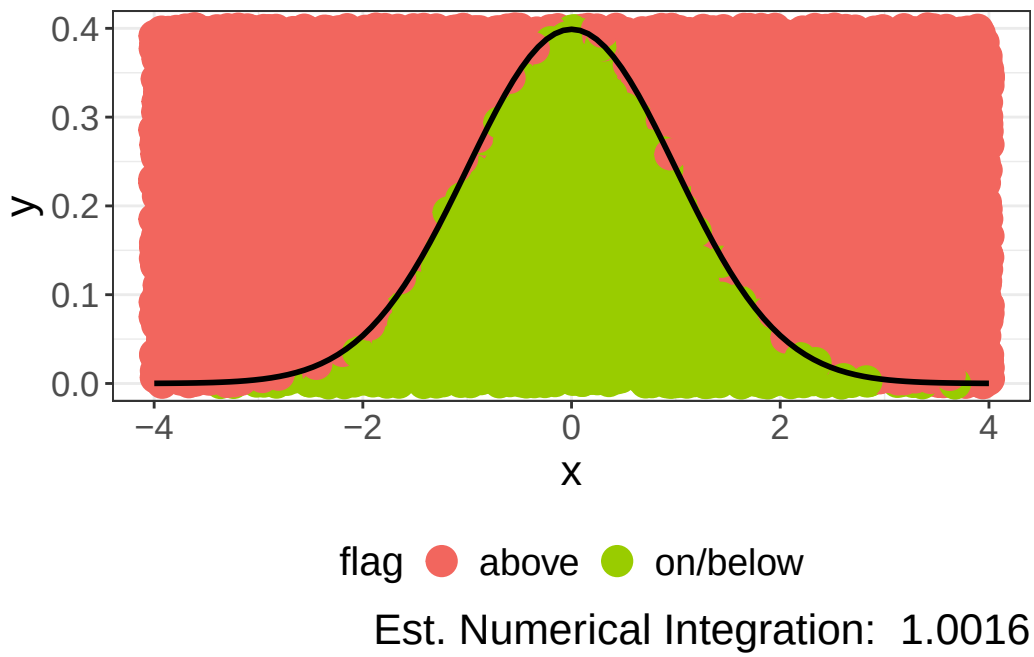

```r
monte_plot(100)
```




```
monte_plot(1000)
```



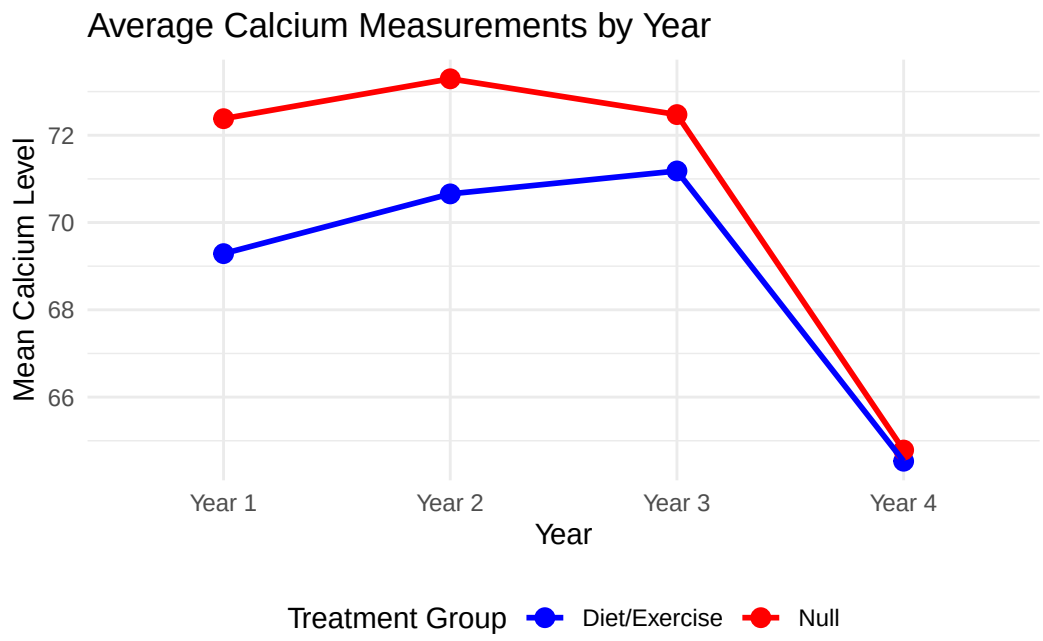

```
monte_plot(10000)
```



- Code for Figure 3

```
#fig-calciumAIGraph
#| fig-cap: "Plan-Informed Prompt Graph of Average Calcium Level Overtime For Each Treat
#| fig-alt: "Line graph showing changes in calcium levels from Year 1 to Year 4 for Diet
#| fig-pos: "H"
#| fig-width: 5

library(tidyverse)

# 1. Import data
calcium <- read.csv("C:\\Users\\03sta\\Downloads\\calcium.csv", header = TRUE)

# 2. Rename columns (first 4 = null group, next 4 = diet group)
names(calcium) <- c("null_year1", "null_year2", "null_year3", "null_year4",
                    "diet_year1", "diet_year2", "diet_year3", "diet_year4")

# 3. Convert from wide → long
calcium_long <- calcium %>%
  pivot_longer(
    cols = everything(),
    names_to = "group_year",
    values_to = "calcium"
  )
```

```r
# 4. Separate treatment group and year
calcium_long <- calcium_long %>%
  separate(group_year, into = c("treatment", "year"), sep = "_")

# 5. Clean treatment names
calcium_long <- calcium_long %>%
  mutate(
    treatment = recode(treatment,
                       "null" = "Null",
                       "diet" = "Diet/Exercise"),
    year = recode(year,
                  "year1" = "Year 1",
                  "year2" = "Year 2",
                  "year3" = "Year 3",
                  "year4" = "Year 4")
  )

# 6. Remove NA rows
calcium_long <- calcium_long %>% drop_na()

# 7. Summary table: mean calcium per treatment per year
calcium_summary <- calcium_long %>%
  group_by(treatment, year) %>%
  summarise(mean_calcium = mean(calcium), .groups = "drop")


library(ggplot2)

ggplot(calcium_summary, aes(x = year, y = mean_calcium,
                            color = treatment, group = treatment)) +
  geom_point(size = 3) +
  geom_line(linewidth = 1) +
  scale_color_manual(values = c("Diet/Exercise" = "blue",
                                "Null" = "red")) +
  labs(
    title = "Average Calcium Measurements by Year",
    x = "Year",
    y = "Mean Calcium Level",
    color = "Treatment Group"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")
```

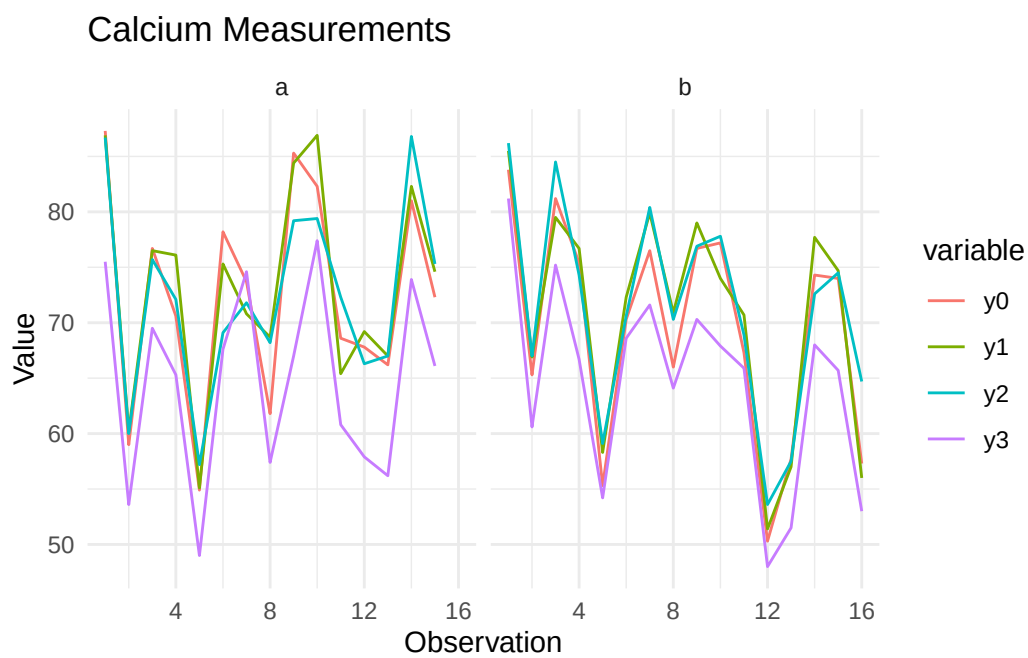


- Code for Figure 4

```
#fig-genAIGraph
#| fig-cap: "Generic Prompt Graph of Obeservation on Calcium Measurements Between Two Gr
#| fig-alt: "Line graph showing calcium measurements with four fluctuating lines for var
#| fig-pos: "H"
#| fig-width: 5

library(tidyverse)

# Read the calcium dataset
calcium <- read.csv("C:\\Users\\03sta\\Downloads\\calcium.csv")

# Fix duplicate column names
names(calcium) <- c("y0_a","y1_a","y2_a","y3_a",
                    "y0_b","y1_b","y2_b","y3_b")

# Reshape to long format
calcium_long <- calcium %>%
  mutate(row = row_number()) %>%
  pivot_longer(
    cols = -row,
```

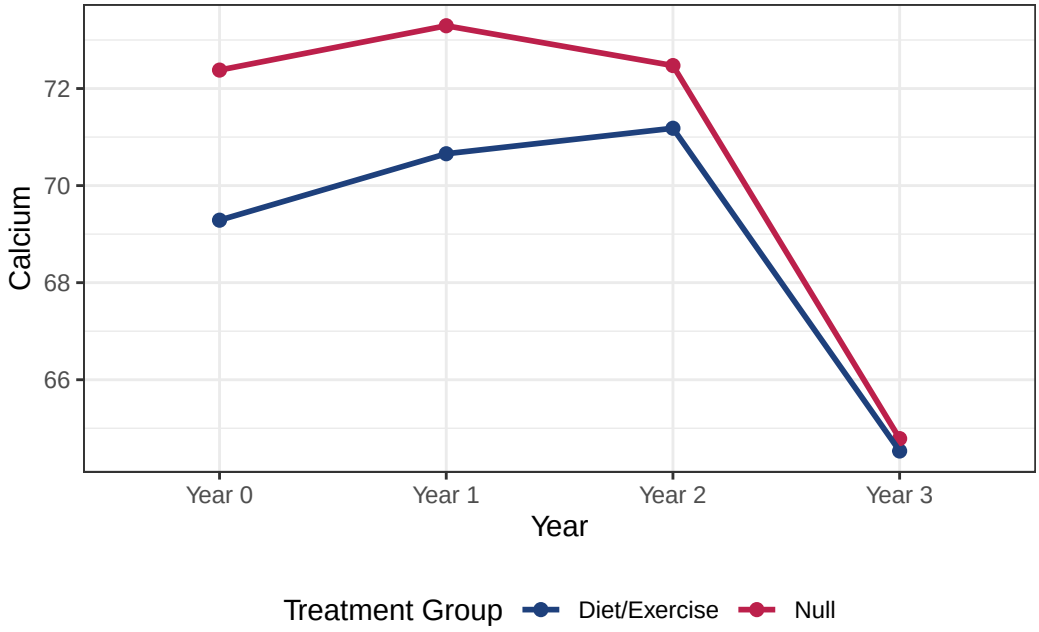
```r
    names_to = c("variable","group"),
    names_sep = "_",
    values_to = "value"
  )

# Basic line plot
ggplot(calcium_long, aes(x = row, y = value, color = variable)) +
  geom_line() +
  facet_wrap(~ group) +
  labs(
    title = "Calcium Measurements",
    x = "Observation",
    y = "Value"
  ) +
  theme_minimal()
```

```
Warning: Removed 4 rows containing missing values or values outside the scale range
(`geom_line()`).
```



- Code for Figure 5

```
#fig-calciumGraph
#| fig-cap: "Line Graph for Average Calcium Level Overtime For Each Treatment Group by J
#| fig-alt: "Line graph showing changes in calcium levels from Year 0 to Year 3 for Diet
```

```
#| fig-pos: "H"
#| fig-width: 5

#Tidying the Calcium Data & Create a line Plot

#Goal 1: Wrangle and tidy the Calcium data.
#Step 1. Import the required packages and data set to analyze.
library(tidyverse)
library(knitr)
library(ggplot2)
library(scales)

##Step 1a. Use read.csv() to check the table for the imported csv file.
calciumRaw <- read.csv("C:\\Users\\03sta\\Downloads\\calcium.csv")

##Step 1b. Check the data set by using View() and str().
#View(calciumRaw)
#str(calciumRaw)

#Step 2. Rename the columns that helps to distinguish between null group and diet group.
calciumCleaned <- calciumRaw |>
  rename( null_year0 = y0, null_year1 = y1, null_year2 = y2, null_year3 = y3, diet_year0

#Step 3. Use pivot_longer() to change the wide format to long format.
  pivot_longer( cols = c(null_year0, null_year1, null_year2, null_year3, diet_year0, die
                names_to = "group_year", values_to = "calcium_levels" ) |>

#Step 4. Separate the combined column names into treatment group and year.
separate_wider_delim( group_year, delim = "_", names = c("treatment_group", "year") ) |>

#Step 5. Rename the values appropriately using the mutate() and recode() from dplyr pack
  mutate( treatment_group = dplyr::recode( treatment_group, "null" = "Null", "diet" = "D

##Step 5a. Remove the rows that contains N/A.
  drop_na(calcium_levels)

#Step 6. Check and revise the code.
#View(calciumCleaned)
#str(calciumCleaned)

#Goal 2. Create a create a line plot that compares the average calcium measurements for
#Step 1. Create a summary table that calculates each treatment group at each year.
```

```r
calciumSummary <- calciumCleaned |>
  group_by(treatment_group, year) |>
  summarise( mean_calcium = mean(calcium_levels),
             .groups = "drop"
             )

#Step 2: Create the framework and specify the mappings.
ggplot(data = calciumSummary, mapping = aes( x = year, y = mean_calcium, color = treatme

#Step 3: Create the content of the graph.
  geom_point(size = 2) + geom_line(size = 1)+ labs( x = "Year", y = "Calcium", color = '

#Step 4: Create the graph's scales using different colors.
  scale_color_manual( values = c( "#1E407C", "#BC204B" ) ) +

#Step 5: Create the legends of the graph.
  theme_bw() + theme(legend.position = "bottom")
```